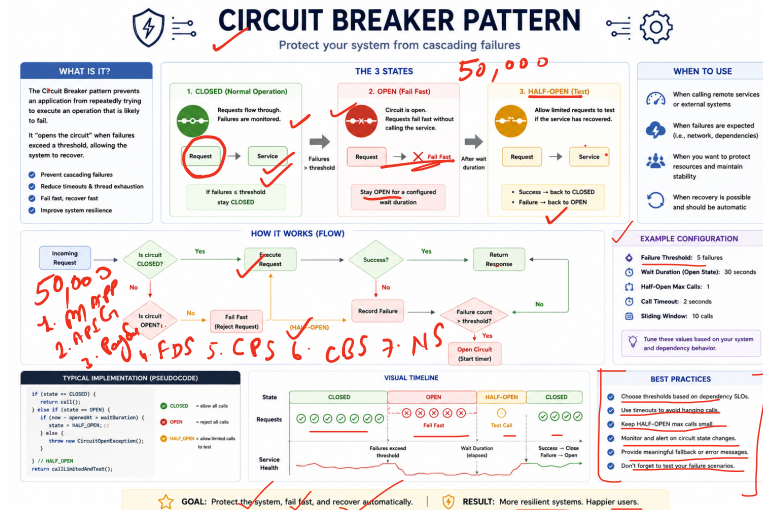
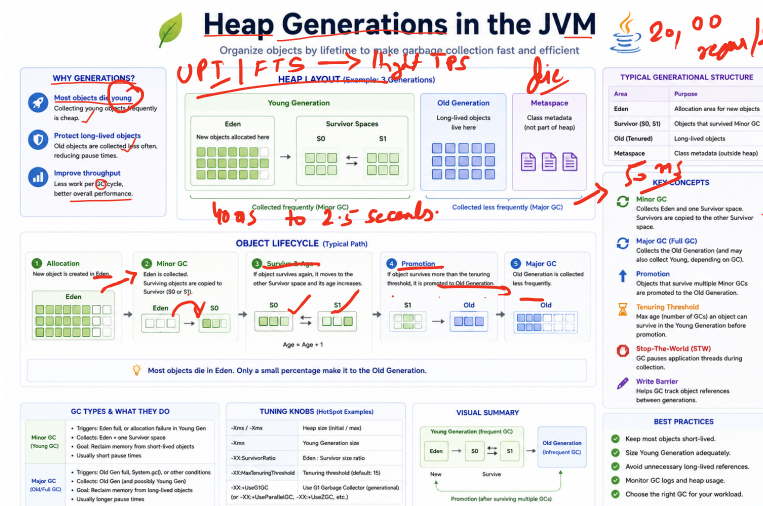




DR (DC)
Transaction Service
↓
fraud Service
100ms
xw Error
Ravi
6m
Emi
Schedule
Emi Service
Core Bank SOAP API



Heap Object
Light
Ob created
Young Gen
Survivors
Promoted old obj.

"UPI payment API experienced latency spikes during peak traffic because JVM spent excessive time scanning the entire heap for short-lived request objects."

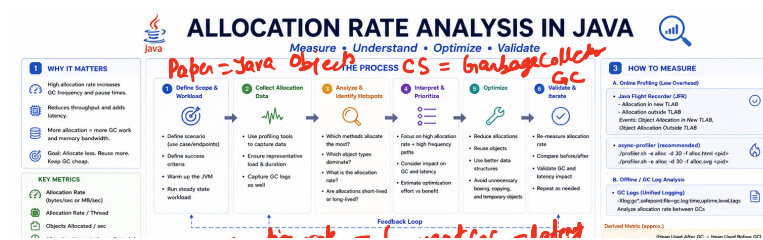
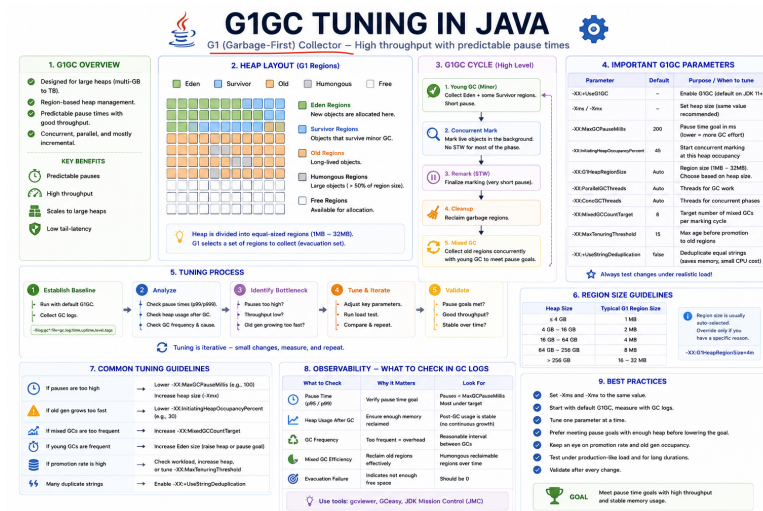
"Long-lived customer session objects were repeatedly scanned during frequent collections, increasing GC overhead and reducing throughput."

"Card authorization latency increased from 35ms to 3 seconds due to promotion pressure and repeated Full GC caused by leaked transaction objects."

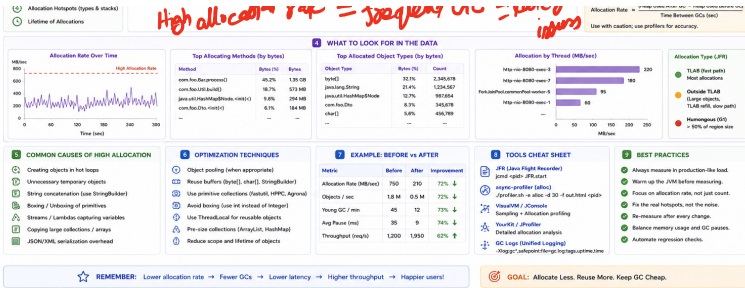
- Solution:
- Heap tuning
 - Leak fixes
 - GC monitoring
 - Manage properly your generation

-- Weak Generational Hypothesis (Most object die young)

G1GC
Salary Day → UPI Traffic Explains
2AM
PR
FR
OWTT
PO DT
31m
Traffic
25000 marks
UPI
CC Pay
sch
wax Risk
min shan
G1GC
young - old



gg diff rate > drain rate.
water overflow.
→ objat rate > gc clamp.
+ many

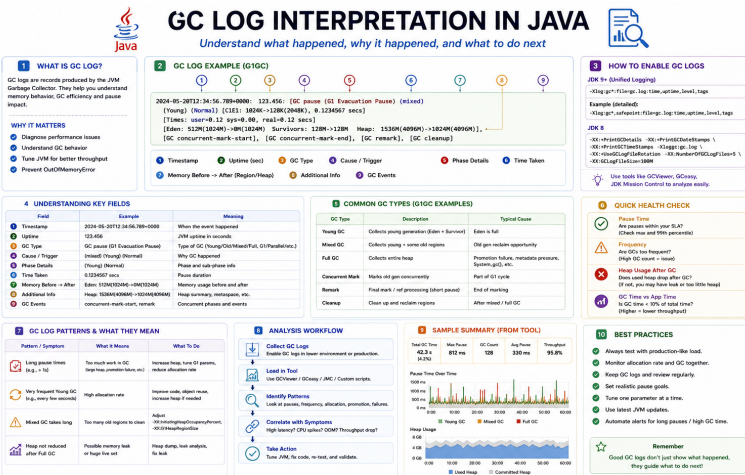


Allocation rate = amount of memory allocated per second

MB/sec

MB/sec

Payment Service - Heap: 4MB Allocated rate 1.6MB/sec



```
public class AllocationExplosionDemo {
    public static void main(String[] args) {
        for (int i = 0; i < 10_000_000; i++) {
            Transaction tx = new Transaction(
                "TXN" + i,
                "CUST" + i,
                1000
            );
            if (i % 2_000_000 == 0) {
                System.out.println(txn);
            }
        }
    }
}
```

```
public class BetterAllocationDemo {
    public static void main(String[] args) {
        StringBuilder builder = new StringBuilder();
        for (int i = 0; i < 10_000_000; i++) {
            builder.append("PAYMENT-")
                .append(i);
            if (i % 2_000_000 == 0) {
                System.out.println(builder);
            }
        }
    }
}
```

```
Static class PaymentRequest {
    Txnid
    customerId
    Amount
}
```

```
public class PureAllocationDemo {
    static class PaymentRequest {
        long txnId;
        long customerId;
        double amount;
    }
}
```

```
PaymentRequest(long txnId, long customerId, double amount) {
    this.txnid = txnId;
    this.customerId = customerId;
    this.amount = amount;
}

public static void main(String[] args) {
    for (long i = 0; i < 50_000_000; i++) {
        PaymentRequest req =
            new PaymentRequest(i, i, 1000.0);
        process(req);
    }

    static void process(PaymentRequest req) {
        if (req.amount == -1) {
            System.out.println("Impossible");
        }
    }
}
```

GOAL: Short pauses, stable throughput, and consistent user experience. Measure → Understand → Optimize → Validate → Repeat